Nearbytes: A Local-First Architecture for Post-Cloud Computing

The Nearbytes Group*

April 27, 2026

Version 0.1. This whitepaper is a work-in-progress version and will evolve together with the specifications, the prototype, and the app, following a "release early, release often" approach.

Abstract

Nearbytes is a local-first, friend-to-friend, data-centric architecture for persistent, collaborative, encrypted data spaces and applications. It stems from immutable encrypted blocks and asks how mutable shared state can be built over durable replicated data, without concentrating naming, storage, and communication in one service, and without depending on the liveness of any particular process. Its answer is to treat state as authenticated replay over replicated data. A secret opens a hub. Signed events define a shared append-only history. Files, chat, profiles, and future applications appear as projections over that history rather than as separate storage universes. This paper describes the system model, the core primitives, the current implementation profile, the replay architecture, and the storage and transport consequences of that design.

1 Introduction

Much of modern computing is still organized around a set of recurring practical questions that users should not have to ask at all. Where is that file? On this machine, on an external disk, on another computer, on my phone? Which copy is the current one? Why does moving among my own devices still feel like exporting, re-uploading, or manually syncing data that is already mine? How do I share this folder with the right people, whether they are nearby or across the globe, without falling back to email attachments, slow chat uploads, fragile Bluetooth pairing, failing proprietary links, or obscure cable transfers? Why is backup still a separate chore rather than a natural consequence of using the system?

The same friction appears in collaboration. How do I work with the people around me without first requiring that all of us adopt the same service, create the same accounts, accept the same terms, and often pay for the same subscription? Why is local collaboration so often harder than cloud collaboration, even when the collaborators are in the same room? Why must durable shared state depend on a central platform being online, on a live account, or on a provider continuing to permit the interaction? Why should basic acts of storage, sharing, backup, and collaboration still present themselves as problems to be managed rather than capabilities that simply exist? Why should ordinary users have to manage relay lists, keep services always online, open ports, or preserve external credentials merely to keep their own data durable and shareable?

Nearbytes makes a strong claim: that these questions can be made wholly *obsolete*. We will do so through a friend-to-friend, viral, and pervasive architecture built from four core primitives:

*Contact: Vincenzo Ciancia vincenzo.ciancia@isti.cnr.it

immutable encrypted blocks, secret-derived hub identity, signed events, and transmission-agnostic synchronization. The point is not to answer each question with yet another service layer. The point is to replace the underlying mechanics so completely that the conditions that generate these questions no longer hold.

Digital storage systems routinely force users to choose among convenience, privacy, and control. In cloud architectures, naming, storage, and communication are concentrated in the same place: the same service stores the bytes, defines the namespace, and mediates access to current state. In traditional peer-to-peer architectures, the opposite tendency appears: the system is organized around running processes and live communication, so when the process stops the system itself recedes or vanishes. Nearbytes begins from a different premise. Replicated data can outlast the process that created it, and if the essential properties of the system are carried by the data encoding itself, the system can persist wherever that data persists.

Nearbytes takes that premise seriously. It begins by reducing storage to durable immutable encoded objects: encrypted blocks that can be copied, retained, and verified independently of any particular machine, session, or route. Once storage is understood that way, a deeper question appears. Where does the living shape of the system arise? How can names, conversations, profiles, and the very sense of one shared hub persist if the underlying data is immutable?

This is the real move of Nearbytes. Meaning does not live inside the blocks alone. It emerges from authenticated history. Files, messages, profiles, and hubs are not stored as final objects waiting to be fetched from a server. They are reconstructed by replaying signed facts over replicated encrypted data. The same construction that lets a file name persist can also make a conversation persist, a profile persist, or an entire *application hub* persist. This is not merely a speculative architecture: an active free and open-source implementation exists in the Nearbytes app repository, where these ideas are already being worked out in code and in an experimental public specifications repository (Nearbytes Contributors, 2026c,b).

The model can therefore be stated simply. Immutable encrypted blocks hold the data. A secret identifies a *hub*, namely a central point of coordination, by deterministically giving rise to a private key, used to sign *events* and a public key which is used as *the identifier* of the hub. By this construction, signed events accumulate the history. Replay reconstructs the current state. In this design, storage defines *what bytes exist*, while history defines *what those bytes mean now*.

From this point of view, content addressing is not a storage trick but a discipline. The system does not begin with mutable containers. It begins with immutable blocks whose integrity can be checked directly, as they are named after a secure hash of their content, and with a separate secret-addressed history whose signed events assign those blocks their current meaning, while proving event authenticity and making unauthorized rewrites detectable.

The system is local-first, friend-to-friend, and data-centric. It is local-first because a user can reopen and interpret a hub from the secret and stored ciphertext, without first asking an account server what exists. It is friend-to-friend because replication can be entrusted to socially chosen peers who store opaque bytes without becoming authorities over the hub. It is data-centric because its resilience is grounded in data encoding rather than in the continuity of any communication protocol. What must survive is carried by the data itself, while higher-level state is reconstructed by replay.

This division also yields practical advantages. Replication can be fully automated and transparent, because preserving a hub does not require preserving a particular session. Backup becomes a natural consequence of copying valid data into additional storage locations. Resilience follows from redundancy rather than from uptime. Since history is retained rather than overwritten, rollback is always available in principle and can be automated in user-facing tools. And in the presence of conflict, the system remains legible: all users can inspect overwrites, verify what happened, and choose a different branch. This is not only a technical property but a democratic principle.

The same separation also makes the system more future-proof. Applications may change, fall out of maintenance, or be replaced, but if the durable properties of state are carried by encoded data and replayable history rather than by one live service, the data remains recoverable in principle beyond the lifetime of any one client implementation.

That is what makes the architecture transport-agnostic without requiring a universal overlay. Nearbytes does not depend on open membership. Its natural setting is smaller and more personal: devices, backups, and trusted contacts exchanging encrypted blocks and signed events over whatever channel is available. Current Nearbytes specifications extend the same model beyond files into chat, profile publication, join links, transport recipes, and multi-source synchronization, all interpreted over the same underlying log (Nearbytes Contributors, 2026b). The application layer is deliberately open: a hub's log is not reserved for one built-in app, but can be projected by as many application protocols as desired, coexisting over the same authenticated history. File management, chat, profiles, sharing, and future protocols are therefore not separate silos but parallel readings and extensions of one hub. One prominent next example is local AI agents, confined to user-defined hubs, where models or agents can read from and append auditable hub records rather than acting through opaque external state (Nearbytes Contributors, 2026a). Such agents could support semantic search across personal and shared hub data, timeline-aware recall and summarization of prior events, context-grounded drafting for messages and documents, proactive organization and triage of files and tasks, and eventually a broader form of AI-assisted digital life that remains inspectable, replayable, and under user control.

The rest of the paper describes the system architecture. Section 2 develops the main use cases and practical pressures that motivate the design. Section 3 states the system model and threat assumptions. Section 4 presents the core primitives. Section 5 records the current implementation profile and the experimental normative specifications that already define canonical storage paths, envelope structure, and synchronization behavior. Section 6 shows how replay turns append-only events into current hub state and why one authenticated history can support several application views. Sections 7 and 8 present two case studies drawn from the current app: file sharing, and chat with profile management. The final section discusses the storage and transport consequences of treating application state as replay over encoded data.

2 Use Cases and Practical Motivation

The motivating claim of Nearbytes is not only that a different storage and collaboration architecture is possible, but that several ordinary computing problems should stop presenting themselves as problems at all. w

2.1 Personal Continuity: Files, Devices, Backup, and Mobility

The most immediate use case is personal continuity across a user's own storage locations and devices. A person should be able to reopen the same hub on a laptop, a phone, a desktop machine, or a removable drive without first remembering which provider currently holds the authoritative copy. In that sense, Nearbytes is meant to make the cloud optional rather than central. A cloud-backed folder may still be one replication route, but it should not be the condition under which data becomes findable or usable.

This also reframes synchronization. In ordinary systems, users often perform a ritual of manual copying, re-uploading, and duplicate hunting just to move among their own devices. In Nearbytes, the intended model is instead that the same hub can be reopened wherever the relevant encrypted blocks and signed history are available. Synchronization becomes exchange of missing blocks and events,

followed by replay, rather than re-establishing a privileged current copy in a provider-controlled namespace.

Backup belongs to the same family of concerns. If valid encrypted content and authenticated history can be replicated opportunistically across additional storage locations, then preservation becomes fully automated, rather than a separate moment. External disks, trusted peers, secondary machines, and other storage locations can all contribute to redundancy. The resulting picture is not one golden backup hidden somewhere else, but socially and materially distributed survival. Resilience follows from redundancy, not from remembering to run a special backup workflow before a failure occurs.

Carrying one's data around is another direct consequence. A phone, a laptop, or a portable disk can hold sufficient encrypted material to reopen a hub locally and continue working. This matters not only for convenience, but for autonomy: important files, records, and conversations remain accessible when a user changes device, loses connectivity, crosses institutional boundaries, or simply does not want current state to depend on a third-party service being reachable.

2.2 Sharing and Collaboration Without Platform Lock-In

The same architecture addresses multi-user collaboration. The practical pressure here is familiar: collaborators may be in the same room or on the same local network, yet the default workflow still routes through a platform, matching accounts, shared terms of service, and often a shared subscription. Nearbytes is meant to support a different default in which people matter more than platforms. The relevant trust relation is between collaborators, participants, devices, and chosen storage peers, not between all of them and a single service operator.

This is valuable at several scales. Small teams can share a project hub for documents, chat, and participant information without dividing those functions among unrelated backends. Families or informal groups can maintain common archives without forcing all members onto the same vendor stack. Research groups and project teams can collaborate locally, across the Internet, or in mixed arrangements, while retaining the same replay discipline and encrypted storage model.

Nearbytes is especially attractive where proximity should help but currently does not. If the relevant devices are on the same local network, or inside the same corporate LAN, synchronization should be able to happen locally and more efficiently than a workflow that first uploads everything outward and only then downloads it back inward. When immutable blocks are already content-addressed and history is append-only, a local exchange can focus on what is missing rather than re-sending an entire mutable directory tree through a remote service. Shared folders, private NAS deployments, removable media, and other passive channels can therefore participate directly in synchronization without being promoted into trusted protocol coordinators. This makes private local synchronization a primary use case rather than an afterthought.

The same point extends to institutions that cannot or will not merge their storage under one provider. Two laboratories, companies, or departments may be able to exchange selected encrypted material and signed updates across narrow trust boundaries even when broad platform-level integration is impossible. The architecture does not remove governance problems, that need to be addressed socially, not mechanically, but it lowers the friction of sharing across institutional boundaries, and puts in plain sight the actual issues that need to be resolved in order to collaborate.

2.3 Transport Diversity: Offline Exchange and Precious Information

Because correctness is tied to verifiable content and signed history rather than to a specific online channel, Nearbytes also supports transmission in network-poor or network-absent settings. This

matters whenever important information must survive movement across machines, sites, or people even if no stable network path is available. A removable drive, a temporary local link, a one-time relay through a trusted contact, or any other opaque-byte transport mechanism can carry hub material. Once the recipient has the encrypted content, the relevant references, and the signed events, replay can resume locally. This is useful for precious information in the broad sense: irreplaceable research material, field notes, legal or administrative archives, shared media collections, medical records, and so on.

Transport diversity also changes the practical meaning of sharing. Email, chat uploads, ad hoc cable transfers, and proprietary short-range protocols are currently treated as different universes. Nearbytes instead treats all transports as possible routes by which the same encrypted blocks and signed history may become available elsewhere.

2.4 Public, Semi-Public, and Social Hubs

Not every hub needs to be narrowly private. The secret-derived identity model also permits deliberately guessable or publicly shared seeds, with the obvious tradeoff that exclusivity and privacy weaken accordingly. This opens a use-case space closer to social networking and rendezvous systems. A publicly known image, phrase, or other widely recognizable artifact could serve as a shared seed and therefore function much like a tag: those who know or reconstruct it can reopen the same public hub and read or append related history.

In that setting, Nearbytes is not replacing the idea of social discovery with strict secrecy; it is supplying a different substrate for it. Small publics, ephemeral communities, event-specific spaces, fandom or topical threads, and public noticeboards can all be modelled as hubs with different secrecy and authority expectations. What changes is not the underlying protocol, but the social meaning of the seed and the governance imposed on top of it.

2.5 Partitioned Analysis, Distributed Computation, and Local AI

Another important class of use cases concerns data that cannot simply be pooled into one shared service because of privacy constraints, intellectual-property barriers, institutional firewalls, or scale. In such settings, collaboration often requires a way to keep raw data local while still coordinating derived results, metadata, annotations, model checkpoints, or agreed summaries. A replayable hub architecture is well suited to this because it separates local possession of data from shared authenticated history about that data.

One example is partitioned collaboration in data analysis. Different parties may hold sensitive corpora that cannot cross administrative or legal boundaries, yet they may still need to share annotations, extracted features, provenance records, partial results, or conclusions. Nearbytes does not remove the underlying policy constraints, but it does offer a substrate in which the shared coordination layer can remain explicit, signed, and replayable while the bulk data stays where it is permitted to stay.

The same applies to distributed training or large-scale model preparation. Datasets may be too large, too sensitive, or too institutionally constrained to merge into one central training store. In such cases, one can imagine local training or preprocessing carried out near each dataset, with only selected derived artifacts crossing boundaries: checkpoints, evaluations, summaries, gradients, or approval records, depending on the policy regime. Nearbytes does not by itself solve all aggregation or verification problems in distributed machine learning, but it does provide a disciplined way to represent what was shared, when it was shared, and under which hub-level authority.

This leads naturally to the AI use case that will be explored in a companion paper (Nearbytes

Contributors, 2026a). If files, chat, profiles, and future application records already coexist in replayable hubs, then AI agents can be confined to explicitly mounted hub material rather than given ambient access to an opaque application backend. Their outputs can appear as summaries, classifications, proposals, patches, or appended records in the same visible history through which human participants already collaborate. In that model, agents are not external oracular services. They are bounded participants in a shared observable changelog.

That matters both for safety and for governance. A local agent can be limited to a chosen hub or subset of hub material. A monitoring agent can inspect the same log for policy violations (be them from other agents, or from humans). Human participants can review proposals before acceptance, or inspect the resulting records afterwards. This makes Nearbytes a promising substrate not only for storage and collaboration, but also for constrained human-centric AI in environments where visibility, locality, and audit matter.

2.6 A Broad but Coherent Design Space

These use cases are diverse, but they are not accidental. Personal continuity, ambient backup, local synchronization, collaboration without platform lock-in, offline transmission, public rendezvous hubs, partitioned data analysis, and auditable local AI all draw on the same structural choice: state is carried by replicable encrypted data plus authenticated history, and current meaning is reconstructed by replay. The rest of the paper demonstrates the architectural design that emerges from this choice, and how it can be implemented in practice.

3 System Model

3.1 Hubs and Storage Locations

Nearbytes distinguishes two levels that are often conflated in other systems. A *hub* is the logical unit of shared state opened from a secret and interpreted as one append-only authenticated history plus associated encrypted content. A hub is typically user-facing and serves the application level, but it is not tied to one fixed application surface: the same authenticated log may be projected by several coexisting application protocols—for example, the same hub can simultaneously serve a chat thread, a shared file folder, and a contact profile, each reading from the same underlying history but interpreting its events according to its own schema. A *storage location* is any physical place where encrypted content and history may be kept: a local folder, a cloud-backed directory, a portable disk, or some other replicated location. A storage location can host several hubs or parts of them, and a hub can be replicated across several storage locations. The two are separate because the hub is defined by its secret and its history, while the storage location is defined by its durability and accessibility.

Once this distinction is made, the main roles become clear. The first is the hub author or writer, who holds the signing authority for that hub and can therefore append valid signed history. In the common case that authority is derived from one secret held by one user, but the same authority may also be shared intentionally when several users derive the same key material from a conventionally shared seed. The second is the storage peer, who keeps copies of ciphertext blocks and event files in one or more storage locations but need not be able to decrypt them. The third is the recipient or reader, who may decrypt, read, import, or mirror part of the shared state if given the necessary references, capabilities, or ciphertext, but who does not thereby gain authority to append valid history to that hub.[1]

[1]The term *participant* belongs more naturally to particular applications layered on top, such as chat or profile

These roles may overlap on one machine, but they need not. A user can be the sole writer on one device while multiple friends or backup devices retain opaque data in separate storage locations. In other cases a group may deliberately share write authority to the same hub while still relying on other devices only as storage peers. A copy operation therefore does not imply trust, and replication does not require a permanent online coordinator. The same person may also occupy different roles with respect to different hubs: author of one, recipient of another, and storage peer for both.

3.2 Trust Assumptions

The system is friend-to-friend in the operational sense that replicas are chosen socially rather than drawn from an open anonymous network. This does not mean peers are trusted with content. Their trust boundary is narrower: they are expected to store bytes and perhaps forward them, while confidentiality and authorship remain protected by encryption and signatures (Popescu et al., 2006). Nearbytes therefore distinguishes availability from authority.

3.3 Threat Model

The design defends primarily against untrusted storage, accidental corruption, and transport substitution. A peer or intermediary that alters a stored block changes its ciphertext hash and is therefore detected at lookup time. A peer that fabricates an event without the correct private key fails signature verification. A peer that withholds data can harm availability, but it cannot silently rewrite authenticated history.

The main residual risk is compromise of the user secret. Because the hub identity and signing authority are both derived from that secret, any attacker who can guess or steal it can reopen the same hub and produce valid updates. Nearbytes thus inherits the ordinary burden of password-based systems: cryptography can protect stored bytes, but it cannot redeem a weak secret (Moriarty et al., 2017). If a seed is intentionally shared among several writers, that shared authority is a feature rather than a breach, but the boundary of trust widens accordingly.

3.4 Related Systems

Nearbytes belongs in a broader family of attempts to build durable shared state from cryptography, append-only history, and socially or structurally constrained replication. Log-structured file systems use append-only update records for efficiency (Rosenblum and Ousterhout, 1991), while systems such as Ivy reconstruct peer-visible state from signed histories (Muthitacharoen et al., 2002). The friend-to-friend literature adds a different pressure: how to obtain durability and sharing without relying on a fully open network or a central authority (Popescu et al., 2006; Li and Dabek, 2006). More broadly, privacy-oriented and socially constrained overlay work such as Freenet helps define the wider design space in which Nearbytes sits, even though Nearbytes is not itself an anonymous overlay network (Clarke et al., 2001). Nearbytes sits in that common terrain, rather than descending from any one of these systems directly.

Modern systems sharpen different parts of that lineage. IPFS makes content-addressed storage central through a generalized Merkle DAG and a global retrieval network (Benet, 2014). Hypercore makes the append-only log itself the primary abstraction, emphasizing signed Merkle-tree verification and sparse replication of streams and datasets (Holepunch, 2026). ERIS, another NLnet-backed effort, is especially relevant at the storage substrate level: it defines content as uniformly sized, encrypted, content-addressed blocks plus a read capability, and thereby sharply separates durable encoded

exchange, than to this core system model, so we avoid using it in this context.

data from any particular transport or application (NLnet Foundation, 2026a). The more recent Earthstar/Willow line, also backed by NLnet and the European NGI programme, pushes closest to the idea of collaborative data spaces: Willow explicitly models hierarchical names, timestamps, and subspaces over content-addressed payloads, while Earthstar instantiates that model for offline-first shared data (Earthstar Project, 2026; NLnet Foundation, 2026b).

Nearbytes is adjacent to all four, but not identical to any of them. Like IPFS, it treats immutable content and cryptographic addressing as foundational. Like Hypercore, it treats append-only authenticated history as the basis of mutable state. Like ERIS, it insists that encrypted durable data should remain valid independently of transport. Like Willow, it points toward hubs in which files, messages, profiles, and future applications can coexist as projections over shared history. Its distinctive emphasis is the combination of secret-derived hub identity, encrypted friend-to-friend replication, and transport-agnostic storage discipline.

4 Core Primitives

4.1 Immutable Encrypted Blocks

At the storage layer, Nearbytes treats encrypted blobs as immutable, content-addressed objects. Once content has been encrypted and stored, it is no longer identified by a mutable filename or a storage location, but by a secure digest of the stored ciphertext. This has two immediate consequences. First, integrity becomes intrinsic: any recipient can verify that the stored bytes are exactly the bytes being referenced. Second, identity becomes location-free: the same object remains the same object wherever it is copied.

This choice separates the existence of content from any particular location. A peer may hold a block, a removable drive may carry it, and a synchronized folder may mirror it, yet the object's identity does not depend on where it was found.

4.2 Secret-Derived Hub Identity

A Nearbytes hub is opened from a secret rather than from a server-assigned identifier. The secret serves as deterministic seed material from which the hub's cryptographic identity is derived. The resulting public key becomes the stable address of the hub, while the corresponding private key grants authority to append valid history. Recovery therefore does not begin with account lookup or server-side naming. It begins with knowledge of the secret and access to some replica of the stored data. The same secret therefore re-opens the same logical event space on any machine that has access to the stored data.

4.3 Signed Events and Replayable State

The third primitive is the signed event. Events bind a payload to the private key associated with a hub, and verification checks that the payload matches the corresponding public key.

Events need not be self-contained. In addition to application data, they may carry *clear-text* references to other event hashes or to payload hashes. Payload references let the system establish that a given content hash belongs to a particular replayable log, or to several logs at once when deduplication and sharing reuse the same stored payload across hubs; so users can express interest in a set of replayable logs (even just implicitly by keeping a hub open) and the system (or storage peers) can route and back up the related payloads automatically, without decrypting them. References to earlier events permit explicit dependency or partial-order relations to be recorded within the history

itself in cleartext. By using this feature, the application level can trade secrecy of ordering-related metadata for proven ordering guarantees on events before decryption.

At the application layer, the payload carries logical metadata such as file name, blob hash, and timestamps. Once signed, these records become append-only history.

4.4 Transmission-Agnostic Synchronization

Nearbytes deliberately avoids binding correctness to a particular network transport. If a receiver obtains a block and an event file, the receiver can verify the block's content hash and the event's signature without contacting an authority server. Implementation can be as simple as keeping payloads all together in a filesystem directory, and each hub log in a directory named after the corresponding public key.

This transmission-agnostic stance is what makes friend-to-friend replication operationally plausible. Replication can happen over cloud-sync folders, external disks, or direct links among devices and contacts, but the protocol invariants remain the same: content is addressed by hash and authority is checked through signatures derived from the hub secret. As a result, merging storage locations is inherently conflict-free at the storage layer: identical payloads coalesce under the same content hash, and authenticated hub histories remain separated by public-key-named directories, so synchronization is a set union of immutable objects rather than an overwrite contest on mutable paths.

Remark 1. *The protocol does not strictly require a human-memorable secret to address a hub. Any valid private key is already sufficient authority to sign hub history, and its corresponding public key is already a stable hub address. Nearbytes emphasizes derivation from a secret because deterministic derivation makes that authority recoverable from remembered or conventionally chosen seed material: the same phrase, mnemonic, image, or other agreed input reconstructs the same keypair and therefore re-opens the same hub. In that sense, the* secret *is best understood as the seed from which hub authority is derived. If the seed is high-entropy and kept private, the hub functions as a private space. If it is deliberately shared within a group, all holders can derive the same authority and write to the same hub. If it is easy to guess or publicly agreed, the hub functions more like a public writable rendezvous point: anyone can reopen it and append history, but privacy and exclusivity are correspondingly weakened. The architecture supports all three cases. What changes is not the protocol, but the security and social meaning of the chosen seed. Furthermore, complex authorization chains can be built on top of the basic model (think e.g. of the application level aggregating multiple hubs and assigning a hierarchical authority structure to them, without requiring the secret of each hub to be shared among the interested parties).*

Remark 2. *Deterministic derivation from a remembered secret is also not mandatory for correctness. Hub keypairs could instead be generated at random from a high-entropy source, as is common in many cryptographic systems. In that model, the burden shifts explicitly to the user* or to the application *to preserve and recover the private key material. One practical design is to store those randomly generated private keys inside a separate Nearbytes volume addressed by a memorable secret and synchronized only to selected devices. That volume then acts as a password wallet or key vault for other hubs: losing it risks loss of signing authority, while protecting and replicating it preserves recoverability without requiring each hub itself to be directly secret-derived.*

5 Implementation Profile and Specifications

The previous sections described Nearbytes at the architectural level. For the current implementation, however, some details are already fixed more concretely in the public experimental specifications repository and in the reference codebase (Nearbytes Contributors, 2026b,c). This matters for three reasons. First, the storage layer is not merely an abstract idea: it is implemented as a canonical object namespace with concrete validity rules. Second, the visible anatomy of a block and of an event is already constrained enough to support storage, routing, retention, and synchronization without requiring decryption. Third, the implementation is deliberately layered so that different host environments may materialize the same Nearbytes objects through different local backends while preserving one logical model.

5.1 Canonical Storage Namespace

The storage layer is currently specified as one canonical logical namespace:

```
blocks/<hash>.bin
channels/<volumeId>/<eventHash>.bin
```

In the current specifications, a block is valid if it is stored under `blocks/<hash>.bin` and the filename hash equals the SHA-256 digest of the block bytes. An event is valid if it is stored under `channels/<volumeId>/<eventHash>.bin`, parses as a valid Nearbytes event envelope, hashes correctly, carries a valid outer signature, and is signed by the public key named by `<volumeId>`. The volume identifier itself is currently specified as the lowercase hexadecimal encoding of the uncompressed 65-byte P-256 public key (Nearbytes Contributors, 2026b).

This gives the storage layer a precise, albeit simple, object model independent of application semantics.

5.2 Anatomy of Blocks and Events

At block level, the current implementation remains deliberately austere. A Nearbytes block is an opaque ciphertext object addressed by the hash of its raw bytes. More structured file content is handled above that level through content descriptors. The current references specifications already distinguish between single-block descriptors and manifest-based multi-block descriptors, so large files need not collapse into one monolithic ciphertext object even though the canonical storage namespace itself remains flat (Nearbytes Contributors, 2026b).

Performance-oriented replay accelerators, including cache structures (e.g., Merkle trees and materialized views), pruning, and reverse-log strategies, are discussed in Remark 3. In this paper they are treated as application-level optimizations over the replay model, not as required storage-layer primitives (Benet, 2014; NLnet Foundation, 2026a; Merkle, 1988; Nearbytes Contributors, 2026b).

At event level, the current opaque-event model is more informative. The hub specifications define an event as a storage-layer object with a visible envelope, an opaque encrypted payload, and a channel-owner signature. The visible envelope currently contains only five things: an event protocol version, the signer public key, a clear-text list of referenced block hashes (`blockRefs`), the ciphertext payload, and the signature. Application semantics are intentionally excluded from the clear-text envelope: filenames, command kinds, timestamps, chat content, identity records, and other higher-level meanings live inside the encrypted payload and are interpreted only by the appropriate subsystem reader after authentication and decryption (Nearbytes Contributors, 2026b).

This separation is what lets the storage layer do useful work without peeking inside application data. The current meta-storage specifications require all decisions about retention, pruning, routing, and retrieval of hub data from local and peer-owned storage locations to depend only on configured sources, canonical paths, volume identities, and the visible `blockRefs` list. In other words, the storage layer is already designed to discover and preserve dependencies from the event envelope alone.

5.3 Storage Layering and Host-Dependent Backends

The implementation does not assume that every Nearbytes deployment looks like a desktop filesystem. The experimental storage-integration specifications currently separate four layers: storage location, storage provider, storage reconciliation, and application layer. A storage location is a concrete local Nearbytes-compatible root. A provider refreshes or publishes one such root for an external system such as MEGA or another managed share. Storage reconciliation decides how several roots participate in one Nearbytes storage graph. The application layer then builds file, chat, join, and sharing workflows over that reconciled graph (Nearbytes Contributors, 2026b).

Storage in Nearbytes is always subject to expansion. A "dumb" storage host may literally store and retrieve Nearbytes blocks and events from a local writable filesystem directory. A browser host may back the same logical paths with IndexedDB. A future native or server-backed host may materialize them through a database or document store. The shared-path storage specification maintains a foundational system invariant: that all data pieces are named after their hash. To implement this, fresh data must become visible – and named after its hash – only after a complete write. Duplicate writes of identical bytes may be treated as no-ops (Nearbytes Contributors, 2026b).

5.4 Incremental Synchronization

The current implementation is currently more specific about synchronization than the specifications alone might suggest. For instance, in LAN-based transport, newly observed events and blocks are advertised over long-lived control channels. The receiver decides whether the referenced object is already present; raw bytes move only if the object is missing. After a missing event is fetched and validated, the receiver inspects its visible `blockRefs` list and requests only the missing referenced blocks (Nearbytes Contributors, 2026b). This kind of incremental behaviour, in practice, makes local synchronization feel immediate once a peer session is already healthy.

6 Replay and Hub Projections

Current state in Nearbytes is obtained by replaying signed append-only history over immutable encrypted content. Nothing at the storage layer is renamed, edited, or deleted in place. What changes is the state that a reader materializes after verifying and interpreting the accumulated events. This replay rule is the architectural center of the system. Immutable blocks establish what encrypted content exists. Signed events establish how that content should be interpreted now.

Remark 3. *Full replay is needed for new participants to join and may be used as a recovery mechanism after failures. At the same time, replay can be significantly optimized at the application level without changing core semantics. As an illustrative example, when inspecting history in reverse, a file-oriented reader may skip creation and update steps for objects that are deleted later. Likewise, application-specific snapshots can serve as checkpoints so replay starts from a recent state instead of traversing the entire history. Cache structures such as Merkle trees and materialized*

views (filesystems, databases, or indexes) can also be constructed on demand during replay. These techniques improve performance, but they remain implementations of full replay semantics rather than alternatives to them. One should consider the full event log replay as the executable specification of the wanted system behaviour, and all optimized synchronization strategies as implementations of that specification.

The file case shown in Figure **??** presents the model in its simplest form:

1. A writer encrypts a document, stores the ciphertext as immutable content, and appends a signed event stating that the logical name `draft.txt` refers to that content.
2. Later, the writer appends another event stating that the same logical object should now be presented as `paper-draft.txt`.
3. Later still, the writer appends a deletion event for that logical object.

Replay does not modify the stored ciphertext. It reads the three events in order and derives the current file view: first the file exists under one name, then under another, and finally it is absent from the live directory view. The mutable directory is therefore not a primitive object. It is a projection over history.

This example is intentionally simple. It is enough to show the replay model, but it does not exhaust what a filesystem built on the same architecture can do. The current example uses whole-file operations, while a richer filesystem could append commands of the form "update block X of file Y with content Z" in order to support genuinely block-level writes without abandoning replay as the mechanism by which current state is derived.

6.1 From a File View to a Hub View

Files are the easiest projection to recognize, but they are not special at the architectural level. A file event names content. A chat record contributes to a conversation. A profile record publishes participant information. Each of these adds a different kind of fact to the same append-only history.

This is the key step beyond a conventional file protocol. Once the hub is the primary object, the file manager becomes one reader among others. The same hub can be read as a directory, a conversation, a participant list, or a bundle of shared references, depending on which protocol vocabulary is applied during replay.

6.2 Illustrative Scenario

Consider a small project hub shared by two collaborators. Its history contains four appended facts:

1. a file event introducing `outline.pdf`,
2. a chat message saying "please review section 2,"
3. a profile update changing one participant's displayed affiliation, and
4. a second file event replacing `outline.pdf` with a newer version.

From those same four facts, replay can materialize several coherent views. A file reader shows the latest version of `outline.pdf`. A chat reader shows the review request in sequence. A profile reader shows the participant's current public description. A combined interface may present all three at once.

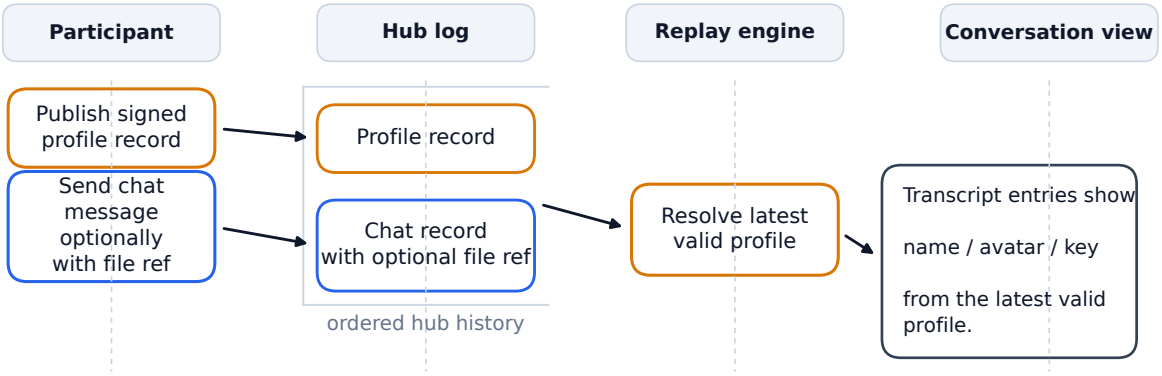

Figure 1: Chat and profiles are projections over the same hub log, not separate systems.

This hub-level reading keeps the architecture small while leaving the application surface open-ended. New protocol families do not require a new storage ontology as long as they can be expressed as replayable records. Transport choices are likewise secondary: a hub may arrive through removable media, local-network exchange, or provider-backed replication, but those are only routes by which the same encrypted content and signed history become available.

7 Case Study: Chat and Profiles

The current version of the Nearbytes app implements simple file managmenet, chat and profile managemnet, in order to demonstrate that the same hub can carry non-file state without introducing a different storage substrate.

In the current design, chat is reconstructed from appended records in the hub log. Profile publication is reconstructed the same way.

One short example is useful to clarify the mechanism. Consider a hub history that contains three appended records:

1. Alice publishes a profile record with her current display name.
2. Alice sends a chat message in the same hub.
3. Alice later publishes an updated profile record.

Replay of the chat records yields the conversation. Replay of the profile records yields Alice's latest public profile. A chat interface can then present the message together with the latest valid profile information for its sender. The profile change does not rewrite the earlier message, and the message does not depend on a central identity directory.

Chat may also carry file references, and profiles may evolve independently of message history, yet both remain readable because they are projections over the same append-only substrate.

8 Conclusions

Nearbytes reduces storage to a small set of rules. Content is encrypted into immutable blocks and addressed by ciphertext hash. A user secret deterministically opens a hub identity. Signed events

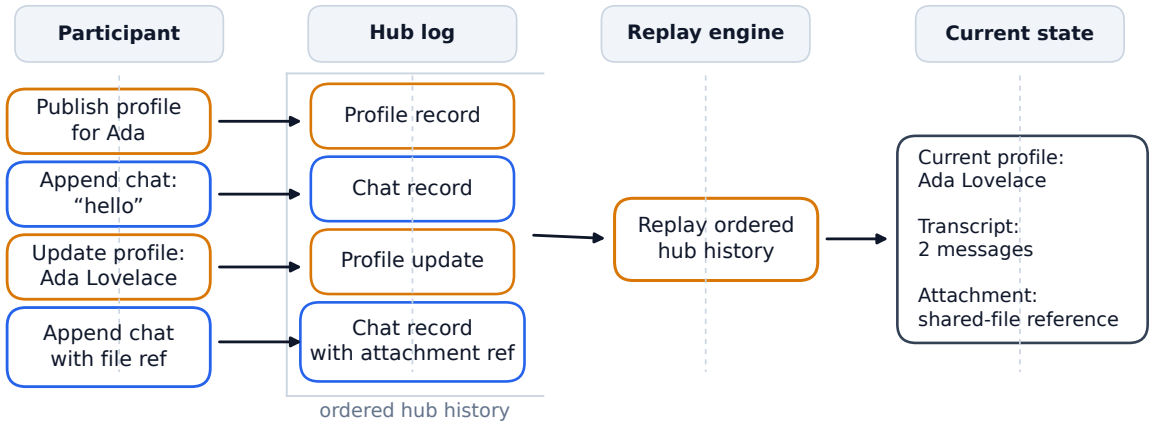

Figure 2: A simple replay example: profile records and chat messages coexist in one append-only hub history.

assign meaning to stored content. Replay reconstructs current state. The central achievement of the design is therefore not replication by itself, but the construction of persistent named and collaborative hubs over immutable data .

That is what gives the system its character. Nearbytes is local-first because a hub can be reopened from the secret and stored data. It is friend-to-friend because replication can be delegated to socially chosen storage peers rather than to an open network. It is data-centric because immutable content is the base layer and application state is derived from authenticated history.

A durable hub can be built over content-addressed encrypted data if meaning is treated as signed, replayable history. If the properties that matter are carried by the encoded data itself, then those properties can survive the disappearance of any particular process, device, or service. That is the paradigm Nearbytes proposes.

References

Juan Benet. IPFS: Content addressed, versioned, P2P file system. Protocol Labs Research report, 2014. URL https://research.protocol.ai/publications/ipfs-content-addressed-versioned-p2p-file-system/.

Ian Clarke, Oskar Sandberg, Brandon Wiley, and Theodore W. Hong. Freenet: A distributed anonymous information storage and retrieval system. In *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability*, volume 2009 of *Lecture Notes in Computer Science*, pages 46–66. Springer, 2001. doi: 10.1007/3-540-44702-4_4. URL https://link.springer.com/chapter/10.1007/3-540-44702-4_4.

Earthstar Project. Willow data model. Protocol specification, 2026. URL https://willowprotocol.org/specs/data-model/. Accessed 2026-04-03.

Holepunch. Hypercore. Project documentation, 2026. URL https://github.com/holepunchto/hypercore. Accessed 2026-04-03.

Jinyang Li and Frank Dabek. F2F: Reliable storage in open networks. In *Proceedings of the 5th*

International Workshop on Peer-to-Peer Systems (IPTPS '06), 2006. URL https://pdos.csail.mit.edu/~jinyang/pub/iptps-f2f.pdf.

Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology—CRYPTO '87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988. doi: 10.1007/3-540-48184-2_32.

Kathleen Moriarty, Burt Kaliski, and Andreas Rusch. PKCS #5: Password-based cryptography specification version 2.1. RFC 8018, January 2017. URL https://www.rfc-editor.org/info/rfc8018.

Athicha Muthitacharoen, Robert Morris, Thomer M. Gil, and Ben Chen. Ivy: A read/write peer-to-peer file system. In *Proceedings of the 5th Symposium on Operating Systems Design and Implementation*, pages 31–44. USENIX Association, 2002. URL https://www.usenix.org/conference/osdi-02/ivy-readwrite-peer-peer-file-system.

Nearbytes Contributors. Safeguards for local ai in nearbytes. Companion manuscript in preparation, 2026a. Forthcoming companion paper on local AI over replayable hubs.

Nearbytes Contributors. Nearbytes specifications. GitHub repository, 2026b. URL https://github.com/nearbytes/nearbytes-specifications. Accessed 2026-04-17.

Nearbytes Contributors. Nearbytes app. GitHub repository, 2026c. URL https://github.com/nearbytes/nearbytes-app. Accessed 2026-04-03.

NLnet Foundation. Encoding for robust immutable storage (eris). Project page, 2026a. URL https://nlnet.nl/project/ERIS/. Accessed 2026-04-03.

NLnet Foundation. Willow sync. Project page, 2026b. URL https://nlnet.nl/project/WillowSync/. Accessed 2026-04-03.

Bogdan C. Popescu, Bruno Crispo, and Andrew S. Tanenbaum. Safe and private data sharing with turtle: Friends team-up and beat the system. In *Security Protocols*, volume 3957 of *Lecture Notes in Computer Science*, pages 213–220. Springer, 2006. doi: 10.1007/11861386_24. URL https://research.vu.nl/en/publications/safe-and-private-data-sharing-with-turtle-friends-team-up-and-bea.

Mendel Rosenblum and John K. Ousterhout. The design and implementation of a log-structured file system. In *Proceedings of the Thirteenth ACM Symposium on Operating Systems Principles*, SOSP '91, pages 1–15. ACM, 1991. doi: 10.1145/121132.121137. URL https://dl.acm.org/doi/10.1145/121132.121137.